

Bangladesh University of Business and Technology



ASSIGNMENT

Assignment No : 01
Course Code : CSE 232
Course Title : Algorithms Lab
Assignment Topic : Lab Mid Assignment

Submitted By:

Name : Kawsar Hoque
ID No : 20255103270
Intake : 55
Section : 07
Program : B.Sc. Engg. in CSE

Submitted To:

Name : Sumona Yeasmin

(Lecturer)
Dept. of CSE
**Bangladesh University of Business
& Technology**

Date of Submission : 08/07/2026

Problem 1: Huffman Coding - Line by line explanation

Here I am explaining the Huffman coding code line by line, in my own way. I try to write what every line is doing so it become easy to follow.

Section 1: Header Files and Constant

```
#include <stdio.h>
```

this line is bringing the standard input output library. we need this because printf() function come from here, and we use printf to show the output on screen.

```
#include <stdlib.h>
```

this line is bringing standard library, and inside this malloc() function is available. we need malloc to take memory for our node whenever we create a new one.

```
#define MAX_TREE_HT 100
```

here we make a constant name MAX_TREE_HT and its value is 100. this number is just showing what is the maximum height our huffman tree can go. later we use this number to fix size of the array which is storing the 0 and 1 code.

Section 2: Node Structure (MH_Node)

```
struct MH_Node {  
    char character;  
    unsigned int frequency;  
    struct MH_Node* l, * r;  
};
```

this is making a structure name MH_Node, every single node of our huffman tree is build from this structure. it is holding 3 thing inside.

character - this is storing the actual letter, like 'a' or 'b'.

frequency - this is storing how many time that letter is coming in our data.

l and r - these two are pointer, pointing to left child and right child of this node in the tree.

Section 3: Min Heap Structure (M_Heap)

```
struct M_Heap {  
    unsigned size;  
    unsigned space;  
    struct MH_Node** array;  
};
```

this structure is representing our min heap. min heap mean the node with smallest frequency always stay on the top. it have 3 field.

size - how many node is currently sitting inside heap right now.

space - the max number of node this heap is able to hold.

array - a pointer which is pointing to array, and this array is holding all the node pointer of the heap.

Section 4: Create New Node - newNode()

```
struct MH_Node* newNode(char character, unsigned int frequency)
```

this function is making one brand new node. it take character and frequency as input parameter.

```
struct MH_Node* temp = (struct MH_Node*) malloc(sizeof(struct MH_Node));
```

here we are asking memory using malloc, enough memory for one MH_Node, and we keep the address inside pointer temp.

```
temp->l = temp->r = NULL;
```

this line set left child and right child of new node to NULL, mean right now this node have no child.

```
temp->character = character;
```

```
temp->frequency = frequency;
```

these 2 line is just saving the character and its frequency value inside our new node.

```
return temp;
```

this send back the new node (temp) to whoever call this function.

Section 5: Create Min Heap - createM_Heap()

```
struct M_Heap* createM_Heap(unsigned space)
```

this function is making a new empty min heap. it take the max size as input.

```
struct M_Heap* M_Heap = (struct M_Heap*) malloc(sizeof(struct M_Heap));
```

this line is taking memory for the heap structure itself, using malloc.

```
M_Heap->size = 0;
```

size is set to 0 because at start heap is completely empty.

```
M_Heap->space = space;
```

this save the max capacity that was given to the function as parameter.

```
M_Heap->array = (struct MH_Node**) malloc(M_Heap->space * sizeof(struct MH_Node*));
```

this is making the actual array that will store node pointer. size of this array depend on how many node the heap can hold.

```
return M_Heap;
```

this return the new empty heap we just made.

Section 6: Swap Two Nodes - swapMH_Node()

```
void swapMH_Node(struct MH_Node** a, struct MH_Node** b)
```

```
{
    struct MH_Node* t = *a;
    *a = *b;
    *b = t;
}
```

this small function is swapping position of 2 node. t is a temporary pointer which hold one node while swap is happening, this is normal swap trick using 3 line. this function get used inside heapify, so smaller node can move to correct position.

Section 7: Heapify - M_Heapify()

```
void M_Heapify(struct M_Heap* M_Heap, int idx)
```

```
{
    int smallest = idx;
    int l = 2 * idx + 1;
    int r = 2 * idx + 2;
```

this function is fixing the min heap rule starting from position idx. first it think current position idx is the smallest one. then it find left child position (2*idx+1) and right child position (2*idx+2).

```
    if (l < M_Heap->size && M_Heap->array[l]->frequency < M_Heap->array[smallest]->frequency)
        smallest = l;
```

this check if left child is existing and if its frequency is smaller than current smallest. if yes then left child now become the smallest.

```
if (r < M_Heap->size && M_Heap->array[r]->frequency < M_Heap->array[smallest]->frequency)
    smallest = r;
```

same checking happen for right child too. if right child frequency is smaller, it become new smallest.

```
if (smallest != idx) {
    swapMH_Node(&M_Heap->array[smallest], &M_Heap->array[idx]);
    M_Heapify(M_Heap, smallest);
}
}
```

if smallest is not same as idx anymore, that mean heap rule is broken here. so we swap the two node, and call M_Heapify again but now from smallest position, this way it keep fixing downward till everything correct.

Section 8: Check Heap Size - isSizeOne()

```
int isSizeOne(struct M_Heap* M_Heap) {
    return (M_Heap->size == 1);
}
```

this function just check if heap have only 1 node remaining. if yes it return 1 (true), if no it return 0 (false). we need this check to know when our huffman tree building is complete.

Section 9: Extract Minimum Node - extractMin()

```
struct MH_Node* extractMin(struct M_Heap* M_Heap)
{
    struct MH_Node* temp = M_Heap->array[0];
```

this function is removing the node with smallest frequency from heap. in min heap the smallest always sit at position 0, so first we save that into temp variable.

```
M_Heap->array[0] = M_Heap->array[M_Heap->size - 1];
--M_Heap->size;
```

the last node of heap is moved to position 0 to fill the gap, then heap size is reduce by 1 since one node just got removed.

```
M_Heapify(M_Heap, 0);
return temp;
}
```

M_Heapify is called from position 0 so heap order become correct again. lastly the removed node (temp) get returned.

Section 10: Insert into Heap - insertM_Heap()

```
void insertM_Heap(struct M_Heap* M_Heap, struct MH_Node* MH_Node)
{
    int i = M_Heap->size - 1;
    ++M_Heap->size;
```

this function is putting a new node inside the heap. i start from the last position, then heap size get increase by 1 because we are adding one more node.

```
while (i && MH_Node->frequency < M_Heap->array[(i - 1) / 2]->frequency)
{
    M_Heap->array[i] = M_Heap->array[(i - 1) / 2];
    i = (i - 1) / 2;
}
```

this loop keep moving the new node upward as long as its frequency is less than its parent frequency. this is how the min heap rule stay correct even after we add new node.

```
M_Heap->array[i] = MH_Node;
}
```

once correct spot is found, the new node get placed right there.

Section 11: Build the Heap - buildM_Heap()

```
void buildM_Heap(struct M_Heap* M_Heap)
{
    int n = M_Heap->size - 1;
    int i;
    for (i = (n - 1) / 2; i >= 0; --i)
        M_Heapify(M_Heap, i);
}
```

this function take all node already put inside array and arrange them into a proper min heap. it start from last non leaf node, then go backward till index 0, calling M_Heapify on every one. after this whole array become valid min heap, smallest one at top.

Section 12: Print Binary Code Array - printArr()

```
void printArr(int arr[], int n)
{
    int i;
    for (i = 0; i < n; ++i)
        printf("%d", arr[i]);
    printf("\n");
}
```

this function print the binary code of one character. it go through the array one by one and print each value which is 0 or 1. once all digit is printed it go to new line.

Section 13: Check Leaf Node - isLeaf()

```
int isLeaf(struct MH_Node* root)
{
    return !(root->l) && !(root->r);
}
```

this check if a node is leaf node or not. leaf node mean it have no left child and no right child, both NULL. only leaf node is holding a real character, so this checking is important.

Section 14: Create and Build Heap - createAndBuildM_Heap()

```
struct M_Heap* createAndBuildM_Heap(char character[], int frequency[], int size)
{
    int i;
    struct M_Heap* M_Heap = createM_Heap(size);
```

this function is creating the heap and also filling it up with node, both together. first empty heap is created with given size.

```
    for (i = 0; i < size; ++i)
        M_Heap->array[i] = newNode(character[i], frequency[i]);
    M_Heap->size = size;
```

loop run for each character, and for every one it make a new node and directly place inside heap array. after loop, heap size is set equal to total size.

```
    buildM_Heap(M_Heap);
    return M_Heap;
}
```

buildM_Heap is called so this array become proper min heap order, then this ready heap is returned.

Section 15: Build Huffman Tree - buildHuffmanTree()

```
struct MH_Node* buildHuffmanTree(char character[], int frequency[], int size)
{
    struct MH_Node *l, *r, *top;
    struct M_Heap* M_Heap = createAndBuildM_Heap(character, frequency, size);
```

this is the most important function, it is the one actually building full huffman tree. first the min heap get created with all node ready inside.

```
    while (!isSizeOne(M_Heap))
    {
        l = extractMin(M_Heap);
        r = extractMin(M_Heap);
```

this loop run until only 1 node remain in heap. every time it run, it take out the two node which have smallest frequency, these become l and r.

```
        top = newNode('$', l->frequency + r->frequency);
        top->l = l;
        top->r = r;
```

a new node top is created with character '\$' which is not real character, just a placeholder. its frequency is sum of l and r frequency, and l, r become its left and right child.

```
        insertM_Heap(M_Heap, top);
    }
    return extractMin(M_Heap);
}
```

this new combined node get inserted back to heap, and loop keep repeating. when only 1 node left, that is the root of full huffman tree, and it get returned.

Section 16: Print Huffman Codes - printCodes()

```
void printCodes(struct MH_Node* root, int arr[], int top)
{
    if (root->l)
    {
        arr[top] = 0;
        printCodes(root->l, arr, top + 1);
    }
```

this function is going through the tree and finding code of each character. if there is a left child, it save 0 in the array position top, then call itself again for left side with top+1.

```
    if (root->r)
    {
        arr[top] = 1;
        printCodes(root->r, arr, top + 1);
    }
```

if there is right child, it save 1 instead, then again call itself for right side.

```
    if (isLeaf(root))
    {
        printf("%c: ", root->character);
        printArr(arr, top);
    }
}
```

when function reach a leaf node, that mean real character found, so it print that character then call printArr to print full code collected so far in the path.

Section 17: HuffmanCodes Function - HuffmanCodes()

```
void HuffmanCodes(char character[], int frequency[], int size)
{
    struct MH_Node* root = buildHuffmanTree(character, frequency, size);
    int arr[MAX_TREE_HT], top = 0;
    printCodes(root, arr, top);
}
```

this function is joining everything together. first it call buildHuffmanTree to get the complete tree and its root. then it make empty array arr for storing the code path, top start from 0. lastly it call printCodes starting from root so every character code get print.

Section 18: Main Function - main()

```
int main()
{
    char arr[] = {'a', 'b', 'c', 'd', 'e', 'f'};
    int frequency[] = {5, 9, 12, 13, 16, 45};
```

this is where program start running. it make a list of 6 character and matching frequency for each of them.

```
    int size = sizeof(arr) / sizeof(arr[0]);
```

size get calculated automatically by dividing whole array size by size of single element, so we don't have to count manually.

```
    HuffmanCodes(arr, frequency, size);
    return 0;
}
```

HuffmanCodes function is called with these value, it will build tree and print final code for every character. return 0 mean program end successfully with no error.

Problem 2: Alternative Implementation of Huffman Coding

For this problem I did not use min heap like problem 1. Here I use simple array and linear search method to find the two smallest frequency node every time, instead of using heap structure. This is a different method to build the same huffman tree, and it is more easy to understand because there is no heapify function needed.

Idea of this method

I keep all node inside a normal array. Every time I need the two node with smallest frequency, I just search whole array one by one to find them (this is called linear search, not heap search). After I find the two smallest, I combine them into one parent node, remove those two from array, and put the new parent node back inside array. I keep doing this until only one node left, that is the root of the tree. After tree is ready, I go through tree and give code to every character, and same time I calculate total bits and average code length.

Full Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 100

struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
};

int totalBits = 0;
int totalChars = 0;

struct Node* createNode(char ch, int freq) {
    struct Node* node = (struct Node*) malloc(sizeof(struct Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

int findMin(struct Node* arr[], int n, int skip) {
    int minIdx = -1;
    int i;
    for (i = 0; i < n; i++) {
        if (i == skip) continue;
        if (minIdx == -1 || arr[i]->freq < arr[minIdx]->freq)
            minIdx = i;
    }
    return minIdx;
}

struct Node* buildTree(char chars[], int freq[], int n) {
    struct Node* arr[MAX];
    int count = n;
    int i;
    for (i = 0; i < n; i++)
        arr[i] = createNode(chars[i], freq[i]);
```

```

while (count > 1) {
    int i1 = findMin(arr, count, -1);
    int i2 = findMin(arr, count, i1);

    struct Node* parent = createNode('$', arr[i1]->freq + arr[i2]->freq);
    parent->left = arr[i1];
    parent->right = arr[i2];

    struct Node* newArr[MAX];
    int k = 0;
    for (i = 0; i < count; i++) {
        if (i != i1 && i != i2)
            newArr[k++] = arr[i];
    }
    newArr[k++] = parent;
    for (i = 0; i < k; i++)
        arr[i] = newArr[i];
    count = k;
}
return arr[0];
}

void traverse(struct Node* root, char code[], int depth) {
    if (root->left == NULL && root->right == NULL) {
        code[depth] = '\0';
        printf("%c : %s\n", root->ch, code);
        totalBits += root->freq * depth;
        totalChars += root->freq;
        return;
    }
    code[depth] = '0';
    traverse(root->left, code, depth + 1);
    code[depth] = '1';
    traverse(root->right, code, depth + 1);
}

int main() {
    char chars[] = {'a', 'b', 'c', 'd', 'e', 'f'};
    int freq[] = {5, 9, 12, 13, 16, 45};
    int n = sizeof(chars) / sizeof(chars[0]);
    char code[MAX];

    struct Node* root = buildTree(chars, freq, n);

    printf("Character : Huffman Code\n");
    traverse(root, code, 0);

    printf("\nTotal bits in encoded message = %d\n", totalBits);
    printf("Average code length = %.2f\n", (float)totalBits / totalChars);

    return 0;
}

```

Explanation

struct Node - this is holding character, its frequency, and pointer to left and right child, same idea as problem 1 but I did not keep any separate heap struct this time.

totalBits and totalChars - these two are global variable, I use them to add up the bits while going through tree, so I don't need any extra loop after.

createNode() - this just make one new node, give it memory, set character and frequency, and make left right child NULL. same work as newNode in problem 1.

findMin() - this is the different part compare to problem 1. instead of heap, this function is doing plain linear search on the array to find index of node with smallest frequency. skip parameter is used so when we already found first smallest, we can search again while ignoring that index, to find second smallest.

buildTree() - this function first make a node for every character and put in array arr. then in while loop, as long as more than 1 node is left, it find two smallest node using findMin, combine them into a parent node with character '\$' and frequency equal to sum of both. then it make a new array without those two node, add the parent instead, and copy this back to arr. this keep happening until only 1 node stay, and that one is returned as root of tree. this array shifting part is the new idea here, in problem 1 the heap was doing this job automatically but here I am doing it manually with simple array copy.

traverse() - this function go down the tree same idea as printCodes in problem 1, adding '0' for left move and '1' for right move. but here, when it reach leaf node, along with printing character and code, it also add up total bits (frequency multiply by depth, since depth is same as code length) and total character count into the global variable. this way average and total bits get calculated same time while printing code, no separate loop needed.

main() - here same character and frequency array is used like problem 1, so that result can be compare. buildTree is called to get root, then traverse is called to print all the code and collect the stats. at the end it print total bits used for whole encoded message and average code length, calculated by dividing total bits by total character count.

Output of this program

Character : Huffman Code

f : 0

c : 100

d : 101

a : 1100

b : 1101

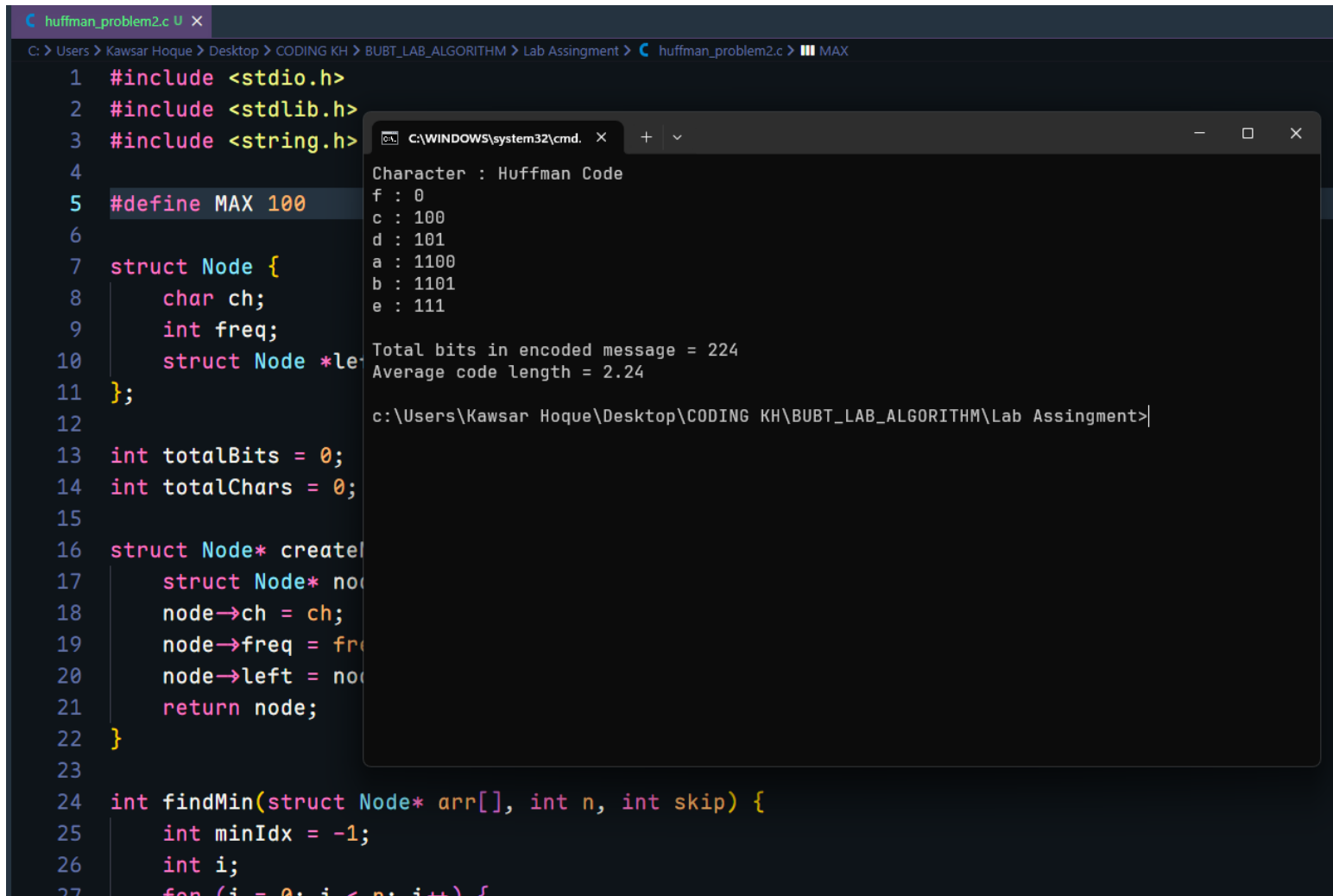
e : 111

Total bits in encoded message = 224

Average code length = 2.24

This result is matching with the sample output given in the assignment sheet, so the code is working correct.

Screenshot of Output (VS Code)



The image shows a Visual Studio Code editor with a C program for Huffman coding. The code is in a file named `huffman_problem2.c`. The program defines a `Node` structure with `ch`, `freq`, and `left` pointers. It includes functions for creating nodes and finding the minimum frequency node. The output window shows the Huffman codes for characters f, c, d, a, b, and e, along with the total bits and average code length.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAX 100
6
7 struct Node {
8     char ch;
9     int freq;
10    struct Node *left;
11 };
12
13 int totalBits = 0;
14 int totalChars = 0;
15
16 struct Node* createNode(char ch, int freq) {
17     struct Node* node = (struct Node*) malloc(sizeof(struct Node));
18     node->ch = ch;
19     node->freq = freq;
20     node->left = NULL;
21     return node;
22 }
23
24 int findMin(struct Node* arr[], int n, int skip) {
25     int minIdx = -1;
26     int i;
27     for (i = 0; i < n; i++) {
```

Output:

```
Character : Huffman Code
f : 0
c : 100
d : 101
a : 1100
b : 1101
e : 111

Total bits in encoded message = 224
Average code length = 2.24

c:\Users\Kawsar Hoque\Desktop\CODING KH\BUBT_LAB_ALGORITHM\Lab Assingment>
```